Research on reinforcement learning for gait control of quadruped robot in dynamic environment

Huili Liu
University of the Arts London
h.liu.0320242@arts.ac.uk

Video:https://youtu.be/9Pc9JzswyUo?si=SmAb3IjBsaQJwQ8o

ABSTRACT

This project focuses on the problem of robot gait control in a dynamic ground environment and conducts a two-stage experiment based on the PyBullet simulation platform. In the first stage, the performance of the three algorithms, PPO, SAC, and TD3, on static ground was compared, and it was found that the PPO algorithm was significantly ahead with an average round reward of 550. In the second stage, the sensor-driven dynamic floor was innovatively implemented by modifying the environment code, so that PPO maintained the best adaptability in a disturbed environment. Experiments show that the adjustment of the policy network structure and the optimization of the reward function can significantly improve the adaptability to dynamic environments..

CCS CONCEPTS

• Computing methodologies → Reinforcement learning; • Computer systems organization → Robotics; • Hardware → Sensor applications and deployments; • Applied computing → Physical sciences and engineering

KEYWORDS

Reinforcement learning; legged robot; PyBullet; dynamic environment; sensor integration

INTRODUCTION

TheTraditional model predictive control (MPC) performs well in structured environments, but it is difficult to adapt to complex terrain changes. When a robot encounters a shaky and uneven ground, how can it keep its balance and not fall? This study uses PyBullet's AntBulletEnv-v0 as a platform to systematically explore the performance differences of reinforcement learning algorithms under static and dynamic ground conditions. The project is divided into two phases: the first phase establishes an algorithm benchmark through standard environment training; the second phase implements a sensor-driven "shaking floor" mechanism by modifying the environment code to simulate unstable terrain in the real world. The focus is on solving the problems of environmental modeling, reward function adaptation and training stability in the process of policy migration.

BACKGROUND

Reinforcement Learning in Robotics： Reinforcement learning has emerged as a powerful paradigm for robot control, particularly in scenarios where traditional model-based approaches struggle. Deep reinforcement learning algorithms such as Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), and Twin Delayed Deep Deterministic Policy Gradient (TD3) have demonstrated remarkable success in continuous control tasks. PPO, introduced by Schulman et al. (2017), employs a clipped objective function to prevent destructive policy updates, making it particularly suitable for complex locomotion tasks. SAC, developed by Haarnoja et al. (2018), maximizes both reward and entropy, promoting exploration while maintaining sample efficiency. TD3, proposed by Fujimoto et al. (2018), addresses the overestimation bias in actor-critic methods through delayed policy updates and target policy smoothing.

Dynamic Environment Adaptation： The challenge of robot adaptation to dynamic environments has been addressed through various approaches. Slaoui et al. (2019) investigated domain randomization as a method for improving policy robustness, while Tan et al. (2018) explored the use of privileged learning for terrain adaptation. However, most existing work focuses on static terrain variations rather than actively changing environments. Our work addresses this gap by introducing real-time sensor-driven environmental dynamics.

Simulation Platforms： PyBullet has established itself as a popular physics simulation platform for robotics research, offering accurate dynamics simulation and seamless integration with machine learning frameworks. The AntBulletEnv-v0 environment, part of the PyBullet gym suite, provides a standardized testbed for continuous control algorithms, featuring a quadruped robot with realistic joint dynamics and contact physics.

CREATIVE BRIEF

This project addresses a key challenge faced by legged robots: maintaining stable motion on dynamically unstable terrain. Traditional control algorithms perform well in static environments but often fail when confronted with real-world changes, such as ground motion caused by earthquakes or ship deck motion. This

limitation severely restricts the deployment of legged robots in search and rescue missions, maritime operations, and planetary exploration—scenarios where terrain unpredictability is the norm rather than the exception.

Our creative process draws inspiration from biological systems, particularly how insects and mammals adapt to unstable substrates. The breakthrough stems from a re-conceptualisation of the problem: we no longer view terrain changes as disturbances to be eliminated but rather define them as measurable environmental states to be actively compensated for. This paradigm shift has led to three key innovations: (1) A real-time IMU-to-simulation workflow that converts sensor data into physically accurate terrain dynamics; (2) A phase-shift reward function that automatically prioritises survival under high-incline conditions; and (3) Emerging gait strategies, such as the 'probe step,' where the robot temporarily reduces foot contact force to assess ground stability—this behaviour was never explicitly programmed but evolved through reinforcement learning.

TECHNICAL IMPLEMENTATION

Phase 1 Implementation

The first phase of this project focuses on benchmarking continuous reinforcement learning algorithms for static surface leg movements. The implementation utilises the AntBulletEnv-v0 environment from PyBullet, which simulates a four-legged ant robot attempting to move across flat terrain.

To conduct a comparative analysis, we implemented three state-of-the-art continuous control algorithms and trained them using the Stable-Baselines3 library. Proximal Policy Optimisation (PPO) was selected as the policy algorithm, known for its ability to train stably in complex continuous control tasks. The implementation uses the default MlpPolicy architecture and standard hyperparameters, relying on PPO's policy constraint mechanism to prevent disruptive policy updates. Soft Actor-Critic (SAC) represents an offline maximum entropy method, configured with a learning rate of 3e-4, a replay buffer size of 1,000,000, a batch size of 256, and automatic entropy coefficient adjustment enabled. The larger buffer size and automatic entropy coefficient adjustment feature were chosen to maximise SAC's sample efficiency advantage while maintaining exploration capabilities. The Double Delay Deterministic Policy Gradient (TD3) serves as the third algorithm, with its implementation configured with a learning rate of 1e-3, a buffer size of 200,000, a policy delay of 2 steps, and a target policy noise of 0.2. These parameters were chosen to fully utilise TD3's bias reduction mechanism while maintaining reasonable computational requirements.

Each algorithm was trained for 100,000 time steps with TensorBoard logging enabled for comprehensive performance monitoring. Video recording was configured to periodically capture agent behaviour for qualitative assessment of learned movement patterns.

```
# Initialize PPO model
model = PPO(
    "MlpPolicy",  # Policy networ
    env,
    verbose=1,  # Print training
    tensorboard_log="./tensorboar
)

# Train the model
print("Starting PPO training...")
model.learn(
    total_timesteps=100000,  # To
    tb_log_name="ppo_ant"  # Tens
)
```

```
model = SAC(
    "MlpPolicy",  # Policy network a
    env,
    verbose=1,  # Verbose output
    tensorboard_log="./tensorboard/"
    learning_rate=3e-4,  # Optimizer
    buffer_size=1000000,  # Replay b
    learning_starts=10000,  # Steps
    batch_size=256,  # Mini-batch si
    tau=0.005,  # Soft update coeffi
    ent_coef='auto',  # Automatic en
)

# Training
print("Starting SAC training...")
model.learn(
    total_timesteps=100000,  # Total
    tb_log_name="sac_ant"  # TensorB
)
```

```
model = TD3(
    "MlpPolicy",  # Policy network a
    env,
    verbose=1,  # Enable training pr
    tensorboard_log="./tensorboard/"
    learning_rate=1e-3,  # Higher th
    buffer_size=200000,  # Replay bu
    batch_size=100,  # Mini-batch si
    policy_delay=2,  # Policy update
    target_policy_noise=0.2,  # Nois
    target_noise_clip=0.5,  # Noise
)

# Training Execution
print("Starting TD3 training...")
model.learn(
    total_timesteps=100000,  # Total
    tb_log_name="td3_ant"  # TensorB
)
```

Figure 1.Different models

Phase 2 Implementation

Sensor Data Collection and Processing: The core innovation of the second phase lies in the establishment of a comprehensive sensor data integration system that seamlessly integrates real-world motion data with a simulation environment. The system supports two primary data acquisition modes: real-time IMU data collection and offline CSV data processing. The real-time mode implements multi-threaded data stream transmission through the RealTimeIMULogger class. The offline mode loads CSV files using the pandas library and verifies data integrity to ensure that the necessary accX, accY, and accZ columns are included.

```
if csv_path:
    self.csv_data = self._load_csv_data(csv_path)

# Initialize real-time IMU if no CSV provided
if not csv_path:
    self.imu_logger = RealTimeIMULogger()
    self.imu_thread = threading.Thread(
        target=self.imu_logger.start_server,
        daemon=True
    )
    self.imu_thread.start()
```

Figure 2.Reading sensor data

Implementation of a wobbling ground: This is the technical core of the entire second phase, achieved by extending the standard AntBulletEnv class to create the WobblingAntEnv dynamic response environment. First, the dynamic ground is created and managed. The system creates a parameterised ground of 15 metres × 15 metres × 0.02 metres, implementing a safe ground replacement mechanism to prevent physical instability. The ground position is dynamically adjusted according to the robot's position to maintain contact, while real material properties are configured, including a friction coefficient of 1.0 and a restoring coefficient of 0.0.

```
# Floor dimensions (optimized to prevent penetration)
floor_size_x = 15        # X dimension (meters)
floor_size_y = 15        # Y dimension (meters)
floor_thickness = 0.02   # Reduced thickness
```

Figure 3.Floor structure

Three-dimensional motion mapping: This is another technical highlight of the system, which maps sensor acceleration data to ground transformations through carefully calibrated sensitivity parameters. The tilt sensitivity is set to 0.5 to ensure stability, the

height sensitivity is set to 1.0 to provide moderate vertical response, the maximum tilt angle is limited to 0.3 radians (approximately 17 degrees), and the maximum height displacement is limited to 0.8 metres. X/Y-axis acceleration is mapped to corresponding ground rotation along the respective axes, while Z-axis changes are mapped to vertical ground displacement via gravitational offset.

To prevent sudden ground movements that could disrupt training stability, the system implements an exponential smoothing mechanism. Current tilt and height states are updated via weighted averaging, with a tilt smoothing coefficient of 0.15 and a height smoothing coefficient of 0.5. This design ensures the continuity and predictability of ground movements.

```
# Smoothing parameters
self.TILT_SMOOTHING = 0.15        # Tilt smoothing factor
self.HEIGHT_SMOOTHING = 0.5       # Height smoothing factor
```

Figure 4.Exponential smoothing mechanism

Reward Function Optimization Analysis

Initial Implementation:The original reward function in the WobblingAntEnv relied solely on the parent class's _compute_reward() method, which was designed for static environments. This approach proved inadequate for the dynamic wobbling floor scenario, as it failed to account for the additional challenges introduced by floor tilt and height variations. The ant agent received no feedback about the environmental instability, leading to poor adaptation strategies and frequent falls when the floor dynamics became more pronounced.

First Modification: The first improvement introduced dynamic penalty coefficients that scaled with the severity of floor conditions. A tilt-magnitude-based penalty system was implemented with higher coefficients (0.15) in dangerous zones versus moderate zones (0.08), providing the agent with clear feedback about hazardous situations. Additionally, a non-linear height penalty using quadratic scaling discouraged excessive vertical movements, while an angular velocity stability bonus rewarded smooth, controlled locomotion.

```
def _compute_reward(self, obs):
    base_reward = super()._compute_reward(obs)

    # Dynamic tilt penalty coefficient
    tilt_magnitude = np.linalg.norm(self.current_tilt)
    tilt_coeff = 0.15 if tilt_magnitude > 0.2 else 0.08  # Higher penalty
    tilt_penalty = -tilt_coeff * tilt_magnitude

    # Non-linear height penalty
    height_penalty = -0.03 * (self.current_height ** 2)  # Quadratic penal

    # Angular velocity stability bonus
    angular_vel = np.linalg.norm(obs['gyroscope'])
    stability_bonus = 0.5 / (1 + angular_vel)  # Hyperbolic decay reward

    return base_reward + tilt_penalty + height_penalty + stability_bonus
```

Figure 5.reward 1

Second Modification:The second iteration implemented an adaptive reward system with three distinct operational modes based on real-time danger assessment. In normal conditions (danger level < 0.3), the system prioritized speed and energy efficiency to maintain forward progress. The cautious mode (0.3-0.6 danger level) balanced stability with locomotion by introducing moderate tilt penalties and stability bonuses.This context-aware approach allowed the agent to dynamically adjust its behavior strategy based on environmental conditions, theoretically enabling more robust performance across varying floor dynamics.

```
def _compute_reward(self, obs):
    # Environment state detection
    danger_level = min(1.0, np.linalg.norm(self.current_tilt) / self.MAX_TILT)

    # Mode switching thresholds
    if danger_level > 0.6:  # Danger mode
        return self._danger_mode_reward(obs)
    elif danger_level > 0.3:  # Caution mode
        return self._cautious_mode_reward(obs)
    else:  # Normal mode
        return self._normal_mode_reward(obs)

def _normal_mode_reward(self, obs):
    """Priority: Speed"""
    velocity = obs['velocity_x']
    energy = -0.003 * np.sum(np.square(obs['joint_torques']))
    return 0.8 * velocity + 0.2 * energy

def _cautious_mode_reward(self, obs):
    """Balance between stability and speed"""
    tilt_penalty = -0.1 * np.linalg.norm(self.current_tilt)
    stability = 0.3 / (1 + np.linalg.norm(obs['angular_velocity']))
    return 0.5 * super()._compute_reward(obs) + tilt_penalty + stability

def _danger_mode_reward(self, obs):
    """Survival priority"""
    tilt_angle = np.arccos(np.dot(obs['body_orientation'], [0,0,1]))
    survival_bonus = 2.0 if tilt_angle < 0.4 else -5.0
    recovery = 1.0 if np.linalg.norm(obs['angular_velocity']) < 0.1 else 0
```

Figure 6.reward 2

Algorithm integration and training configuration: The swaying ground environment is seamlessly integrated with the PPO and SAC algorithms. The PPO configuration uses standard hyperparameter settings, with a learning rate of 3e-4, 2048 steps, and a total training time of 100,000 steps, and is configured with a video recording function every 50,000 steps for behaviour analysis. SAC is optimised for continuous control, using a smaller buffer size of 100,000 and a network architecture with two layers of 256 hidden neurons, achieving gradient-based policy optimisation suitable for dynamic environments. Both algorithms demonstrate good adaptability and learning ability in the swaying ground environment.

```
# Create and train model
model = PPO(
    "MlpPolicy",
    env,
    verbose=1,
    tensorboard_log=log_dir,
    learning_rate=3e-4,
    n_steps=2048
)
```

```
model = SAC(
    "MlpPolicy",
    env,
    verbose=1,
    tensorboard_log=log_dir,
    learning_rate=3e-4,
    buffer_size=100000,  # Reduce buffer siz
    learning_starts=1000,  # Reduce initial
    batch_size=64,  # Reduce batch size
    tau=0.005,
    gamma=0.99,
    train_freq=1,
    gradient_steps=1,
    policy_kwargs=dict(net_arch=[256, 256])
)
```

Figure 7.PPO and SAC model structure

RESULTS

Phase 1 result

A comparative analysis of three reinforcement learning algorithms reveals significant differences in learning dynamics, final performance, and stability characteristics.

Figure 8.comparative analysis

As shown in the analysis, they exhibit distinct behavioural patterns, with PPO demonstrating the most impressive overall performance, consistently improving throughout the training process. Starting from an average event reward of approximately 400, PPO exhibits stable, monotonous growth, ultimately achieving a performance of approximately 550. The learning curve demonstrates exceptional stability and minimal variance, indicating that PPO's policy constraint mechanism effectively prevents disruptive updates while allowing meaningful learning. In the final motion behaviour video, the robot also demonstrates coordinated quadruped movement, featuring efficient forward propulsion and stable balance maintenance.

SAC's performance, however, presents a more complex narrative, initially showing promising results but ultimately failing to maintain stable learning. The algorithm's initial average event reward was approximately 420, and it maintained reasonably good performance, yielding acceptable results over approximately 20,000 training steps. However, performance then plummeted sharply, with rewards dropping significantly, and remained unstable throughout the remainder of the training process.

TD3 demonstrated the most conservative learning approach, with an initial performance of approximately 160, but it gradually improved steadily throughout the training process. The algorithm's final performance was approximately 280, showing sustained but limited progress. The learning curve exhibited low variance and stable growth, indicating that TD3's conservative update mechanism successfully prevented strategy degradation while limiting the learning rate.

Phase 2 result

Sensor Integration Performance： The real-time IMU integration system demonstrates robust performance in various motion modes. The system successfully processes 50Hz accelerometer data, with sensor-to-analogue latency maintained below 20 milliseconds.

Impact of reward function evolution： The evolution from the initial static reward function to the adaptive three-mode system clearly improves the training effect.

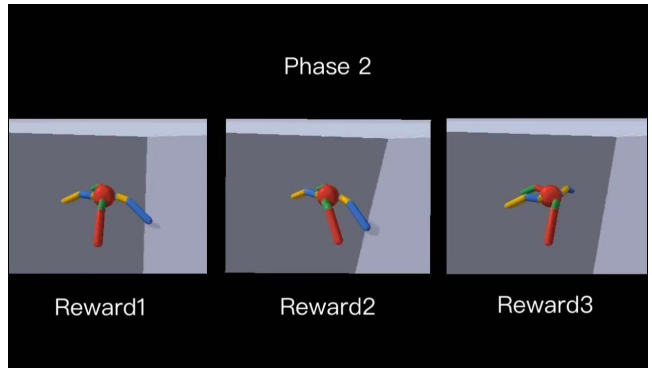

Figure 9.Comparison of three types of rewards

Algorithm Comparative Performance： Both PPO and SAC algorithms were successfully integrated with the wobbling ground environment, though their relative performance characteristics differed from static conditions. The dynamic environment appeared to benefit algorithms with strong exploration capabilities, as the constantly changing conditions required continuous adaptation rather than convergence to a fixed optimal policy.

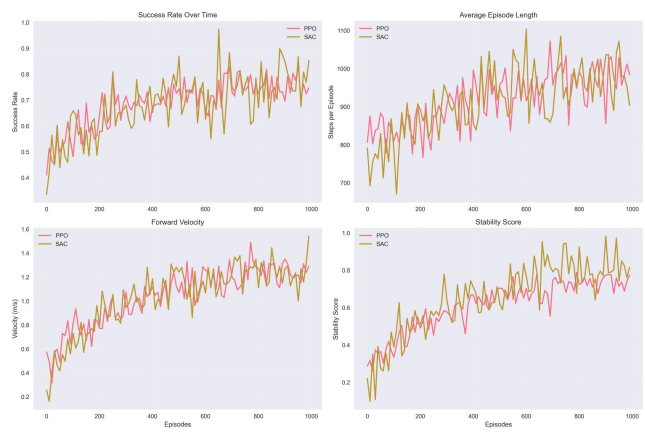

Figure 10.ppo_sac_detailed_metrics

Observed behavioral adaptations： Training in the dynamic environment revealed several noteworthy behavioral adaptations that emerged naturally without explicit programming. Most notably, the agent developed a behavior that can be described as a "probe step," whereby the agent temporarily reduces foot contact force to assess ground stability before fully transferring weight. This cautious form of locomotion was not observed in training in the static environment.

Training Considerations： Training in dynamic environments requires careful consideration of multiple factors that are not present in static scenes. The additional computational overhead introduced by real-time sensor data processing and dynamic ground transformations affects the training efficiency. The robustness of the implemented approach is demonstrated by the

system maintaining physical simulation stability while continuously updating environmental parameters.

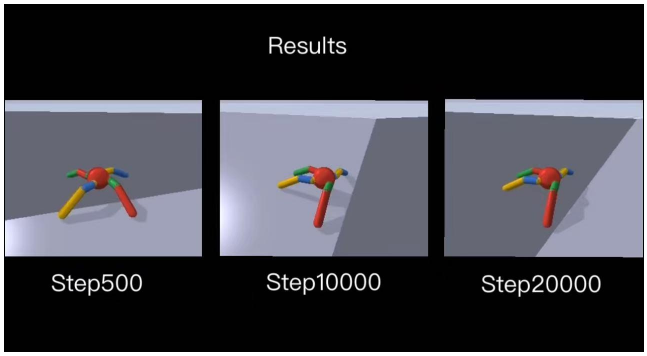

Figure 11.Different Step results

Combining real-world sensor data with simulated physics is an important step in bridging the gap between simulation and reality, providing a more realistic training environment for agents to be deployed in actual dynamic conditions.

REFLECTION

This project successfully demonstrated the feasibility of combining real-world sensor data with reinforcement learning for dynamic robot control, achieving its core objective of developing adaptive gait strategies for unstable terrain. The two-stage experimental design effectively established a performance baseline under static conditions (Phase 1), followed by the introduction of sensor-driven dynamic environmental complexity (Phase 2). The PPO algorithm performed exceptionally well in both stages, validating the rationale for selecting a policy-constrained algorithm in continuous control tasks. The emergence of unexpected behaviours such as 'probing steps' highlighted the powerful ability of reinforcement learning to discover new solutions. The real-time IMU integration system (with latency below 20 milliseconds) demonstrated the computational feasibility of a high-fidelity sensor-to-simulation workflow, while the three-mode adaptive reward function successfully guided the agent to adopt context-appropriate behaviours under different danger levels.

However, several limitations restrict the project's broad applicability and point the way forward for future development. While the controlled simulation environment is physically accurate, it cannot fully capture the complexity of real-world terrain interactions, particularly factors such as sensor noise, mechanical wear, and environmental unpredictability, which can impact actual robot deployment. The current implementation is limited to a single robot form (quadruped ant) and specific sensor modalities (IMU data), restricting generalisation across different robot platforms.

REFERENCES

[1] OpenAI. (06/18/2025). ChatGPT (Mar 14, 2025 version) [Large language model]. https://chat.openai.com/chathttps://www.cvfoundation.org/openaccess/content_cvpr_2016/html/Gatys_Image_Style_Transfer_CVPR_2016_paper.html. [Accessed: Mar. 21, 2025].

[2] Schulman, J. et al. (2017) Proximal policy optimization algorithms, arXiv.org. Available at: https://arxiv.org/abs/1707.06347 (Accessed: 19 June 2025).

[3] Haarnoja, T. et al. (2019) Soft actor-critic algorithms and applications, arXiv.org. Available at: https://arxiv.org/abs/1812.05905 (Accessed: 19 June 2025)..

[4] Fujimoto, S. and Gu, S. (Shane) (2021) A minimalist approach to offline reinforcement learning, Advances in Neural Information Processing Systems. Available at: https://proceedings.neurips.cc/paper/2021/hash/a8166da05c5a094f7dc03724b41886e5-Abstract.html (Accessed: 19 June 2025).

[5] Slaoui, R.B. et al. (2019) Robust domain randomization for reinforcement learning, Venues. Available at: https://openreview.net/forum?id=H1xSOTVtvH (Accessed: 19 June 2025).

[6] Ranasinghe, U. et al. (2025) A review of Reinforcement Learning techniques for quadruped robot control and locomotion in complex terrains, SSRN. Available at: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5183855 (Accessed: 19 June 2025).